Software Requirements Specification

Project: NetSentinel Platform (Network Security Monitoring)
Date: July 9, 2026
Version: 1.0 (Core Modules Plan)

1. Introduction

This Software Requirements Specification (SRS) outlines the architectural and functional requirements for **NetSentinel**, an enterprise-grade real-time Network Security Monitoring (NSM) and Intrusion Detection System (IDS). Traditional intrusion detection platforms are inadequate for lightweight inspection because they typically rely on heavy platform-dependent C kernel drivers (such as WinPcap/Npcap), closed binary engines, and complex multi-tool stacks. NetSentinel resolves this by providing a self-contained, high-performance security monitoring architecture engineered entirely in **Modern Java (Java 21/25 LTS, Spring Boot 3, Project Loom Virtual Threads)** paired with a lightweight web operations dashboard.

The platform bridges the gap between low-level packet capture and operational threat intelligence. It ingests network traffic through dual modes—supporting pure Java binary PCAP dissection and live physical decoy honeypot probes—executes deterministic rule matching alongside statistical flow anomaly detection, persists security events into an ACID-compliant SQLite relational database, and streams live telemetry to a browser dashboard.

2. System Overview & User Roles

NetSentinel is designed as a distributed, decoupled pipeline architecture where five functional modules communicate through frozen JSON contracts and well-defined service boundaries. The backend is orchestrated by Spring Boot 3 utilizing Java 21+ Virtual Threads, with SQLite providing persistent storage and HTML5/JS (Chart.js) delivering client-side telemetry.

2.1 User Classes

- **Security Analyst (SOC Operator):** The primary operational user. Analysts monitor real-time traffic volume graphs, inspect incoming alert feeds, review severity badges (`[HIGH]` , `[MEDIUM]` , `[LOW]`), track malicious source IP frequencies, and use the interactive Live/Pause control to freeze telemetry during incident analysis.
- **Network Administrator:** Administrative users with authority to adjust detection window thresholds, configure active decoy honeypot ports, review SQLite audit logs, trigger PCAP re-evaluations, and purge alert histories.
- **Red-Team Evaluator / Simulated Attacker:** Evaluators or automated harnesses that generate controlled network traffic (such as Nmap port scans, SYN floods, and SSH brute-force attempts) across physical LAN connections or through built-in simulation triggers.

> **Architectural Note:** System components interact via strict, frozen JSON contracts (`Packet JSON` and `Alert JSON`). All public endpoints enforce global Cross-Origin Resource Sharing (CORS) policies, permitting the client dashboard to run either as embedded static resources within Spring Boot or from an independent frontend dev server.

3. Functional Requirements (Core Modules)

Module 1: Packet Capture & Preprocessing (Pure Java)

This module serves as the primary data ingestion boundary, converting raw network frames and binary capture files into standardized Java packet records without native C libraries.

- **Binary PCAP Dissection:** The module shall parse standard libpcap (`.pcap`) files in pure Java, supporting both Big-Endian (`0xA1B2C3D4`) and Little-Endian (`0xD4C3B2A1`) header formats.
- **Physical Decoy Probes:** The system shall bind unprivileged listening sockets across designated decoy ports (`2222` , `8088` , `8888` , `9090`) to intercept real-time connection attempts originating from any external host on the physical network.

Module 2: Rule-Based Detection Engine

This module provides deterministic, signature-based intrusion detection using in-memory sliding time windows.

- **Sliding-Window State Queues:** The engine shall maintain state queues using `HashMap<String, ArrayDeque<Instant>>` to track packet arrival timestamps within configurable sliding windows.
- **Deterministic Signatures:** The system shall detect Port Scans (≥ 15 ports / 5s), SYN Floods (≥ 50 SYNs / 2s), and Brute Force attempts (≥ 10 attempts on sensitive ports / 30s) with a 5-second per-source cooldown timer to prevent alert flooding.

Module 3: Anomaly & ML Detection Engine

This module acts as a secondary behavioral detection layer, analyzing statistical flow deviations that may bypass static rule thresholds.

- **Flow Feature Extraction:** The engine shall aggregate raw packets into bidirectional flows over 5-second windows, calculating packet counts, byte volume, SYN/ACK ratios, and distinct destination ports.
- **Shannon Port Entropy Scoring:** The module shall compute Shannon entropy (`H = -∑ p_i log2(p_i)`), flagging flows exhibiting high entropy (`H > 2.5`) across ≥ 12 ports as automated reconnaissance anomalies (tagged with calibrated confidence 0.70 to 0.99).

Module 4: Central Backend & Storage Orchestration (Spring Boot)

The central orchestrator connecting capture, detection, persistence, and external presentation.

- **Pipeline Coordination & SQLite:** The service shall ingest packet streams, fan out records concurrently to Modules 2 and 3, and persist security events into an ACID-compliant SQLite relational database (`nsm.db`).
- **REST API & 1Hz Metrics:** The backend shall expose frozen REST endpoints (`/api/alerts` , `/api/traffic-stats` , `/api/summary`) and maintain a thread-safe 30-second rolling buffer of packets-per-second.

Module 5: Real-Time Web Operations Dashboard

The user-facing presentation layer delivering live SOC monitoring and interactive threat evaluation.

- **Live SOC Visualizations:** The dashboard shall poll telemetry every 2 seconds, rendering real-time Chart.js line charts, severity badges (`[HIGH]` , `[MEDIUM]` , `[LOW]`), and active attack feeds.
- **Interactive Analyst Controls:** The interface shall provide an instant Live/Pause stream toggle for forensic inspection, and 1-click synthetic attack triggers for evaluation.

4. Database Architecture (Relational & In-Memory Strategy)

NetSentinel employs a hybrid persistence model: durable SQLite relational storage for audit logs and alerts, paired with thread-safe in-memory rolling buffers for high-frequency dashboard telemetry.

Feature Category	SQLite Relational Core (`nsm.db`)	In-Memory Rolling Buffer (RAM)
Data Types	Intrusion Alerts, Sampled Packet Headers, Attacker IP Records.	Rolling Packets/Sec, 1Hz Timestamps, Sliding Window Queues.
Schema Rules	Strict relational schema, primary keys, timestamp indexing.	Circular arrays, thread-safe synchronized lists, FIFO deques.
Connection	Spring `JdbcTemplate` via `sqlite-jdbc` driver.	JVM Heap memory (`AtomicLong` , `Collections.synchronizedList`).
Primary Key Strategy	8-character unique hexadecimal UUIDs (`alert_id`).	In-memory queue index positions (0 to 29).
Concurrency Mode	Write-Ahead Logging (WAL) with synchronized writes.	Non-blocking atomic counters and concurrent collections.

5. Non-Functional Requirements

- **Performance & Latency:** The in-memory rule engine and flow feature extractor must process individual packets in under 1 ms. The Spring Boot API must resolve `/api/traffic-stats` and `/api/alerts` requests in under 15 ms, with portfolio aggregation payloads returning in under 500 ms.
- **Security & Input Sanitization:** All incoming JSON payloads must be validated against expected schemas. Client-side dashboard rendering must sanitize string inputs to prevent Cross-Site Scripting (XSS). Decoy sockets must safely terminate TCP handshakes without leaking operating system internals.
- **Fault Tolerance & Isolation:** A processing failure or format anomaly within Module 3's ML engine must not interrupt Module 2's deterministic rule matching or Module 4's database persistence. In-memory buffers must preserve alert state during transient database locks.
- **Concurrency & Scalability:** Packet ingestion, decoy probe listeners, and synthetic attack generators must execute on **Java 21+ Virtual Threads (`Thread.ofVirtual()`)** to ensure high concurrency without OS thread exhaustion.
- **Portability & Zero Native C:** The complete platform must compile and run on any standard operating system (Windows, Linux, macOS) equipped with Java 21+ without requiring native C compilation, WinPcap, Npcap, or external database daemon installations.